DFX Lab Food/Drink Monitor

User Manual and Technical Reference

DFX Capstone Team | University of South Florida | CIS 4910
Augusto Trufelman | Claudio Fischer | Damien Naha | Felipe Vignatti
Version 2.0 — April 2026

Table of Contents

1. System Overview

The DFX Lab Food/Drink Monitor is an automated compliance tool for the USF Bellini College Design for X (DFX) engineering lab, where food and beverages are strictly prohibited. The system uses one or more cameras, a YOLOv8 computer-vision model, MediaPipe pose landmarks, and a web dashboard so that supervisors can review potential violations in real time from any browser on the local network.

The system has been substantially refactored since its initial release into a clean Python package (dfx/) and a thin entry-point script (dashboard.py). Key improvements over the initial version include:

- Modular package architecture (dfx/) — detection, alerts, camera, motion, training, server, GPU, and settings are each in their own module.
- Two-camera fixed layout — a primary inference camera (Camera 0, Zone G) and a secondary preview camera (Camera 2, Zone F) stream simultaneously.
- MediaPipe Holistic pose landmark detection — wrist-to-mouth 2D proximity replaces the older frame-difference proxy approach for hand-to-mouth detection.
- Persistent dashboard state — zone counts and processed alert IDs survive restarts via dashboard_state.json.
- Detection summary CSV — every alert writes one row per detection to detections_summary.csv for offline analysis.
- Kaggle/local offline training script (train_local_kaggle.py) — independent of the dashboard accept/reject flow.
- VRAM management during training — the live inference model is moved to CPU before fine-tuning begins and restored afterward.
- Configurable custom SNACK classes — chip bags, candy, and vending-machine food are now in the detection vocabulary.

2. System Requirements

Component	Minimum / Recommended
Hardware	NVIDIA Jetson Orin Nano (production) or any Linux/macOS/Windows PC with a webcam
Python	3.9 or newer (enforced by install_project.py)
Camera	V4L2-compatible (Linux) or DirectShow-compatible (Windows); two cameras for full dual-zone operation
Network	Local LAN so operators can reach the dashboard from any browser
Disk	~500 MB for virtual env, model weights, snippets/, and training_data/
Optional	MediaPipe (pip install mediapipe) for strict landmark-based hand-to-mouth detection

3. Repository Layout

The project root after installation looks like:

```
dashboard.py            # entry point: launches HTTP server + camera worker
main.py                 # command-line webcam demo (no HTTP, for quick testing)
install_project.py      # bootstrap script (venv, deps, dirs, model download)
train_local_kaggle.py   # offline training from local or Kaggle datasets
import_and_train.py     # helper: imports a dataset into the training pipeline
requirements.txt        # pip dependencies
run_dashboard.sh        # Jetson wrapper (sets LD_LIBRARY_PATH, activates venv)
run_main.sh             # Jetson wrapper for main.py
start_gpu.sh            # optional: hot-swap best.pt into a running dashboard
alerts.json             # persisted alert log (auto-created)
dashboard_state.json    # persisted zone counts and processed IDs
detections_summary.csv # per-detection CSV log (auto-created)
class_map.json          # class-name→id mapping written after training
dfx/                    # package root
  __init__.py           # re-exports for backward compatibility
  alerts.py             # alert I/O, snippet saving, CSV export, suppression
  camera.py             # camera capture loop, inference, alert state machine
  constants.py          # all tuning constants and class-name sets
  detection.py          # YOLO result parsing, bounding-box drawing
  gpu.py                # Jetson detection, device selection, TensorRT export
  model_loader.py       # YOLO model loading with hot-swap support
  motion.py             # consumption motion scoring + MediaPipe hand-to-mouth
  multicam.py           # CameraPreview + CameraManager for dual-camera layout
  server.py             # HTTP handler (all REST endpoints + MJPEG stream)
  settings.py           # DashboardConfig dataclass, validation helpers
  training.py           # accepted-sample export, fine-tuning worker, hot-swap
snippets/               # cropped bounding-box JPEGs (auto-created)
training_data/          # dataset/ and runs/ directories (auto-created)
assets/dfx_lab_map.png # optional lab layout image for Map view
```

4. Installation

4.1 Quick Start (all platforms)

Run the bootstrap script from the project root:

```
python3 install_project.py
```

The script will:

- Create a .venv virtual environment in the project root.
- Install ultralytics, opencv-python, numpy, and all other dependencies from requirements.txt.
- Create the snippets/, training_data/dataset/images/, training_data/dataset/labels/, and training_data/runs/ directories.
- Create alerts.json as an empty JSON array if absent.
- Download yolov8n.pt if a custom model is not already present.
- On Jetson, generate run_dashboard.sh and run_main.sh with the correct LD_LIBRARY_PATH.

The script is idempotent and safe to re-run.

4.2 Jetson Orin Nano

If CUDA-enabled PyTorch is not already installed system-wide, supply the official NVIDIA wheel:

```
python3 install_project.py --torch-wheel <wheel-url-or-path>
```

> **Note:** The system has been validated on Jetson Orin Nano with JetPack 6. On this hardware always launch via ./run_dashboard.sh so the CUDA library search path is set correctly before the interpreter starts.

4.3 Optional: MediaPipe

MediaPipe enables the strict wrist-to-mouth landmark detector. Without it the system falls back to the heuristic proxy scorer.

```
source .venv/bin/activate
pip install mediapipe
```

5. Launching the Monitor

5.1 Standard launch

```
source .venv/bin/activate   # Linux/macOS
python dashboard.py
```

On Windows, use .venv\Scripts\activate instead of source.

5.2 Jetson launch (recommended)

```
./run_dashboard.sh
```

This wrapper sets LD_LIBRARY_PATH for the Jetson CUDA libraries and activates the virtual environment automatically.

5.3 Recommended Jetson settings

```
python dashboard.py \
  --inference-imgsz 960 \
  --max-inference-fps 4  \
  --fps 6               \
  --jpeg-quality 65
```

> **Note:** Add --no-motion-enabled to further reduce CPU load if the motion heuristics cause dropped frames.

5.4 Command-line-only demo (no web UI)

```
source .venv/bin/activate
python main.py
```

This opens the camera, draws bounding boxes in an OpenCV window, and writes alerts to alerts.json. Useful for local testing without starting the HTTP server.

5.5 Finding the dashboard

After launch the server prints a URL such as http://0.0.0.0:8000. Open any browser on the same network and navigate to http://<device-ip>:8000 to access the live dashboard.

6. Dashboard Walkthrough

6.1 Live Feeds

The top of the page shows two live MJPEG streams: the primary inference camera (Camera 0, Zone A) with annotated bounding boxes, and the secondary preview camera (Camera 2, Zone F). Both cameras are physically mounted upside-down and are flipped 180° in software before display and inference.

6.2 Alert Cards

Below the feeds, alert cards appear in reverse-chronological order. Each card shows:

- Alert ID, timestamp, zone, and detected class names with confidence scores.
- Snippet thumbnails — cropped bounding boxes from the source frame. Clicking a thumbnail shows the full-resolution crop.
- Status buttons: Reviewed (real event, handled), Dismissed (false positive), or Accepted (feed into training).
- A consumption_motion flag and score if the motion heuristics were active when the alert fired.

6.3 Alert States

State	Meaning
New	Unreviewed. Appears at the top of the list.
Reviewed	Operator acknowledged; real event handled.
Dismissed	Marked as false positive or no longer relevant.
Accepted	Confirmed for fine-tuning; snippets are exported to training_data/ at next training run.

State changes are persisted by POST /alerts/manage, which updates alerts.json in place. Full alert history is always preserved; only the status field changes.

6.4 Group by Zone

A toggle on the dashboard collapses the alert list into one block per zone so that operators monitoring a multi-camera setup can focus on a particular area.

6.5 Map View

The Map button loads an image of the DFX lab floor plan (assets/dfx_lab_map.png by default, overridden with --map-image). Zone labels are overlaid so operators can identify which physical camera corresponds to each zone.

6.6 Consumption Stats

The Stats view calls GET /stats/consumption and renders a table showing, for each zone, how many of each food category have been flagged over the current alert log.

6.7 Settings Panel

The Settings panel lets operators change runtime parameters live without restarting:

- Inference image size (160–1280 px; default 640).
- Max inference FPS (0 = unlimited; cap 0–60).
- Stream FPS and JPEG quality for the MJPEG feeds.
- Confidence and IoU thresholds.
- Persistence frames, clear frames, and cooldown seconds for the alert state machine.
- Motion detection toggle and tuning parameters.

A Reset button restores all settings to the values used at launch. Every field is validated and clamped on the server side before it is applied.

6.8 Training Mode

Alerts marked Accepted can be used to fine-tune the model. Press Train in the Training panel to start a background job. The training worker will:

- Suspend live inference to free VRAM.
- Export accepted snippets to training_data/dataset/.
- Run YOLOv8 fine-tuning with Jetson-safe defaults if on Jetson hardware.
- Hot-swap the model at training_data/runs/accepted/weights/best.pt back into the running dashboard.
- Resume live inference.

Training progress is streamed back to the dashboard via GET /train/status.

7. Command-Line Reference

7.1 Camera and Network

Flag	Default	Description
--cam N	0	Primary camera device index.
--host HOST	0.0.0.0	Address to bind the HTTP server.
--port N	8000	Port for the HTTP server.
--zone ZONE	Zone A	Zone label for the primary camera (Zone A–Zone I).
--map-image PATH	assets/dfx_lab_map.png	Path to the lab layout image.

7.2 Streaming

Flag	Default	Description
--fps N	10	Target stream FPS to the browser.
--inference-imgsz N	640	YOLO image size for inference (160–1280).
--max-inference-fps F	0	Cap on YOLO inference rate. 0 = unlimited.
--jpeg-quality N	75	MJPEG JPEG quality (40–95).

Flag	Default	Description
--width N / --height N	0	Resize frame before inference. 0 = native.

7.3 Detection and Alerting

Flag	Default	Description
--conf F	0.55	YOLO confidence threshold.
--iou F	0.40	YOLO IoU threshold for NMS.
--persist-frames N	5	Consecutive frames before alert fires.
--cooldown S	15	Minimum seconds between alerts.
--clear-frames N	15	Empty frames required to re-arm.
--alert-log PATH	alerts.json	Path to the JSON alert log.
--snippets-dir PATH	snippets/	Directory for cropped snippet images.

7.4 Motion Analysis

Flag	Default	Description
--motion-enabled / --no-motion-enabled	enabled	Toggle consumption motion layer.
--motion-window N	12	Frames kept in per-object motion history.
--motion-displacement-threshold F	0.07	Normalized displacement threshold.
--motion-upward-threshold F	0.02	Normalized upward-component threshold.
--motion-hold-seconds F	0.1	Seconds a positive motion signal stays active.

7.5 Training

Flag	Default	Description
--model PATH	yolov8n.pt	YOLO weights file to load at startup.
--train-epochs N	10	Epochs for accepted-sample fine-tuning.
--train-imgsz N	640	Image size for fine-tuning.

8. Troubleshooting

Camera cannot be opened

Verify the camera is connected and not in use by another process. On Linux run ls /dev/video* to confirm the device is visible. Pass the correct index with --cam. On the Jetson the primary camera is index 0 and the secondary is index 2.

Inference too slow on Jetson

Use the recommended Jetson flags from section 5.3. Lower --inference-imgsz to 320 or 256, cap --max-inference-fps to 4 or 2, reduce --fps and --jpeg-quality, and consider --no-motion-enabled which disables the per-frame heuristics on top of YOLO.

No detections for common snacks

The default yolov8n.pt model recognizes only the 80 COCO classes. Items such as chip bags, candy bars, and energy drinks are detected only after the custom SNACK class set is enabled and a fine-tuned model covering those classes is provided via --model.

MediaPipe not available

If mediapipe is not installed the system logs a warning and falls back to the heuristic proxy scorer. Install it with pip install mediapipe inside the virtual environment.

Training fails with CUDA allocator error

train_local_kaggle.py automatically retries on CPU when a CUDA memory allocator failure is detected. For dashboard-triggered training, the VRAM management worker moves the inference model to CPU before the training job begins and restores it afterward; if errors persist reduce --train-imgsz or add more accepted samples before training.